

ROS2 입문 · 15주 대학 강의계획서

Ubuntu 24.04 · ROS2 Jazzy · Gazebo Harmonic

교과목 개요

- 대상: Python 기초 문법·함수·클래스를 학습한 대학생. ROS2 경험은 필요하지 않습니다.
- 운영안: 15주, 주 3시간, 총 45시간. 일반 주차는 개념 1시간 + 실습 2시간, 8·15주는 평가 중심입니다. 학점 인정과 출결 규정은 대학 학칙에 맞춰 확정합니다.
- 수업 외 학습: 매주 예습·복습·과제 2시간 권장. 교과목명, 개설 학과, 담당교수, 강의실과 상담시간은 개설 기관에서 지정합니다.
- 방식: 짧은 개념 설명 → 예제 실행 → 조건 변경 → 결과 관찰 → 개인 기록. 1~13주는 기초와 모델링을 누적하고 14~15주에 통합합니다.

학습 성과

1. ROS2 작업 공간을 구성하고 Python 패키지를 빌드·실행한다.
2. Topic·Service·Action을 요구에 맞게 선택하고 통신 결과를 검증한다.
3. Launch·QoS·Parameter·Namespace로 실행 조건을 구성한다.
4. TF·URDF·Xacro와 RViz2를 이용해 로봇 모델을 설명하고 점검한다.
5. 시뮬레이션의 센서 입력과 제어 출력을 관찰하고 재현 가능한 프로젝트를 제출한다.

평가 계획

- 출석·실습 참여 10%, 주차 과제 25%, 중간 개인 실기 25%, 기말 프로젝트 40%로 총 100%입니다.
- 중간 실기 25점: 빌드·재실행 5, 통신 동작 10, 오류 진단 5, 설명과 증거 5.
- 기말 40점: 요구·설계 5, 구현 동작 15, 검증·오류 분석 10, 재현 문서 5, 개인 설명·기여 5. 팀 점수 35점과 개인 점수 5점을 구분합니다.
- 주차 과제는 재현 가능성 40%, 관찰·해석 40%, 기록 완결성 20%로 평가합니다. 지각 제출·결석 처리는 개강 때 공지합니다.

주차별 수업 계획

각 주 3시간이며, 8주 중간 실기와 15주 최종 시연도 총 45시간에 포함합니다.

1주 · ROS2 개요와 개발 환경 (1~3장)

- 목표: 로봇을 Node와 통신으로 설명하고 Ubuntu·WSL2·ROS2의 역할을 구분한다.
- 실습: 설치 점검 후 ROS2 기본 Talker-Listener를 실행한다.
- 완료 기준: 송신·수신 로그를 대조하고 사용한 OS·ROS 배포판을 기록한다.
- 제출물: 환경 점검표와 설치 오류 해결 기록.

2주 · 터미널·Git·ROS 그래프 (4~5장)

- 목표: 터미널 작업과 ROS 그래프 조회를 수행한다.
- 실습: Turtlesim을 실행하고 ros2 node list, ros2 topic list로 구성 요소를 찾는다.
- 완료 기준: 키보드 조작으로 거북이가 움직이는지 확인하고 Node-Topic 연결을 그린다.
- 제출물: 명령 기록과 ROS 그래프.

3주 · Workspace·Package·Python Node (6~7장)

- 목표: 패키지 생성·빌드·source·실행의 순서를 재현한다.
- 실습: 학습용 별도 패키지를 만들고 maker_basic/simple_node를 참고해 Node를 작성한다.
- 완료 기준: 수정 후 다시 빌드·source하여 바뀐 로그를 확인한다.
- 제출물: 패키지 코드와 새 터미널 재현 절차.

4주 · Topic과 클래스형 Node (8~9장)

- 목표: Publisher-Subscriber와 타이머 실행 주기를 설명한다.
- 실습: maker_basic의 message_pub, message_sub와 class_message_pub, class_message_sub를 비교한다.
- 완료 기준: 메시지 내용·주기 변경 전후 수신 로그를 비교한다.
- 제출물: Topic 실습 코드와 주기 관찰표.

5주 · Launch-DDS-QoS (10~12장)

- 목표: 여러 Node를 실행하고 QoS 호환성을 설명한다.
- 실습: class_message.launch.py와 qos_message_pub, qos_message_sub를 사용한다.
- 완료 기준: QoS 설정을 맞추거나 다르게 한 뒤 수신 여부와 ros2 topic info -v 결과를 기록한다.
- 제출물: Launch 파일과 QoS 비교 보고서.

6주 · 사용자 정의 Message와 Service (13~14장)

- 목표: 연속 데이터와 요청·응답 통신을 구분한다.
- 실습: UserInt.msg, AddAndOdd.srv와 maker_basic의 user_int_pub, user_int_sub, add_server, add_client를 실행한다.
- 완료 기준: 인터페이스 필드와 실제 메시지·응답 값을 대조한다.
- 제출물: 입력·예상 응답·실제 응답 표.

7주 · Executor·비동기·Parameter (15~16장)

- 목표: 콜백 처리와 설정값 분리를 설명한다.
- 실습: `threaded_add_server`, `periodic_status`, `config/status.yaml`, `status.launch.py`를 살펴보고 설정을 변경한다.
- 완료 기준: 동시 요청 처리 로그와 YAML 변경 후 상태 출력 차이를 설명한다.
- 제출물: 실행 시간선과 설정 파일.

8주 · 중간 실기 평가 (1~16장)

- 목표: 환경부터 통신·설정까지 독립적으로 재현한다.
- 실습: 주어진 Topic·Service 요구를 구현하고 Launch 또는 YAML로 실행 조건을 정리한다.
- 완료 기준: 새 터미널에서 재실행하고 의도적으로 주어진 source·설정 오류를 찾아 고친다.
- 제출물: 개인 실기 코드·로그·짧은 설계 설명.

9주 · Action과 작업 상태 (17~19장)

- 목표: Goal·Feedback·Result 및 취소·실패 흐름을 구분한다.
- 실습: `Fibonacci.action`과 `gong_basic/action_server`, `action_client` 8의 성공 흐름부터 관찰한다.
- 완료 기준: Feedback와 최종 결과를 대조한다. Cancel·Abort는 코드의 실제 처리 분기를 확인하고 별도 시험 입력 또는 클라이언트 수정으로 검증한다.
- 제출물: 상태 전이도와 성공·취소·실패 시험 계획 및 수행 여부.

10주 · Namespace와 다중 로봇 (20장)

- 목표: 이름 충돌 없이 두 로봇의 명령을 분리한다.
- 실습: gong_basic/launch/two_turtle.launch.py와 mv_turtle_ns를 사용한다.
- 완료 기준: 한 Namespace에 준 명령이 대상 거북이에만 영향을 주는지 확인한다.
- 제출물: Node-Topic 목록과 기말 프로젝트 제안서.

11주 · TF2 좌표 변환 (21~22장)

- 목표: 부모-자식 좌표계와 변환 방향을 설명한다.
- 실습: tf2_basic의 static_turtle_tf2_broadcaster, dynamic_turtle_tf2_broadcaster, tf_listener를 확인·실행한다.
- 완료 기준: 정적·동적 변환 출력과 움직임의 관계를 비교한다.
- 제출물: TF 트리와 변환 해석 기록.

12주 · URDF·Link·Joint (23~24장)

- 목표: 링크와 관절을 구성하고 JointState와 모델 움직임을 연결한다.
- 실습: two_link_fixed.urdf와 two_link.urdf를 비교하고 ch24_joint_motion.launch.py를 실행한다.
- 완료 기준: GUI 모드를 종료한 뒤 코드 발행 모드로 전환하고 /joint_states 변화와 팔 회전을 확인한다.
- 제출물: URDF 변경본과 관절 관찰 기록.

13주 · Xacro 재사용과 RViz2 검증 (25~26장)

- 목표: 매크로·접두어로 모델을 재사용하고 시각화 오류를 진단한다.
- 실습: reusable_robot.urdf.xacro, prefixed_arm.urdf.xacro, ch25_two_arms.launch.py를 사용한다.
- 완료 기준: Fixed Frame world와 각 Robot Description을 설정해 두 모델의 프레임 충돌 여부를 확인한다.
- 제출물: Xacro 코드와 프로젝트 중간 점검 결과.

14주 · Gazebo 센서 기반 제어 (27장)

- 목표: LaserScan 입력과 주행 명령의 관계를 설명한다.
- 실습: wall_follow_lab 안내대로 wall_room.launch.py와 wall_follow를 실행하고 tests/test_controller.py를 수행한다.
- 완료 기준: /clock:/scan:/cmd_vel의 연결을 확인하고 벽과의 거리·회전 반응을 관찰한다. 합성 검사와 주행 결과를 따로 기록한다.
- 제출물: 조건별 제어 관찰표와 프로젝트 리허설.

15주 · 기말 프로젝트 시연·회고 (종합)

- 목표: 설계·실행·문제 해결 결과를 증거로 설명한다.
- 실습: 2~3인 팀이 통신·모델링·제어 중 선택한 통합 과제를 시연한다.
- 완료 기준: 다른 팀이 README 절차로 재현하고 요구 충족·실패 조건을 확인한다.
- 제출물: 코드·실행 안내·시연 증거·개인 기여 및 회고.

기말 프로젝트 운영

10주 제안 → 13주 중간 점검 → 14주 리허설 → 15주 시연으로 진행합니다. 기본 예제 실행에 최소 한 가지 기능·설정 변경을 더하고, 변경 이유와 전후 결과를 설명합니다.

- 통신형: Namespace로 분리한 Turtlesim 제어. 로봇별 명령 분리와 통신 방식 선택 이유를 검증합니다. 20장의 완성 답안은 제공되지 않으므로 통합 부분은 직접 구현합니다.
- 모델형: Xacro로 재사용 가능한 팔을 구성하고 JointState 변화와 TF·RViz2 결과를 대조합니다.
- 제어형: wall_follow_lab의 제어 조건을 변경하고 두 가지 이상의 센서·환경 조건에서 반응을 비교합니다. 실물 로봇 적용은 범위에 포함하지 않습니다.

발표는 팀당 시연 5분과 질의 3분을 기준으로 하며, 팀 수가 많으면 동시 시연 부스를 사용합니다. 제출 README에는 환경, 설치·빌드, 실행·종료, 기대 결과, 알려진 한계를 포함합니다.

실습 준비와 운영

- 환경: Ubuntu 24.04, ROS2 Jazzy, Python 3, colcon, Git, VS Code. Windows 사용자는 WSL2와 WSLg를 먼저 점검합니다. 후반 실습에는 RViz2와 Gazebo Harmonic이 필요합니다.
- 준비 순서: 무료 샘플 2~3장의 OS-ROS2 설치 → README Quick Start의 저장소 복제·rosdep-4개 패키지 빌드 → class_message 통신 확인 → 27장 안내의 Gazebo 의존성 설치·wall_follow_lab 별도 빌드 순서로 진행합니다.
- 모든 실행은 Ubuntu Bash에서 진행합니다. 새 터미널마다 source /opt/ros/jazzy/setup.bash와 source ~/ros_ws/install/setup.bash를 실행합니다. 작업 공간을 다른 경로에 만들었다면 경로를 바꿉니다.
- 강사는 개강 전 학생과 동일한 PC 환경에서 Turtlesim, RViz2, Gazebo GUI와 /clock:/scan:/cmd_vel 연결까지 예행연습합니다. 여러 학생이 같은 네트워크를 쓰면 실습별 ROS_DOMAIN_ID를 배정하고 한 학생의 터미널끼리는 동일하게 맞춥니다.
- 준비물: 개인 개발 PC, 인터넷, 코드 편집기, 실행 로그 저장 공간. 시뮬레이션 중심으로 운영하며 실제 로봇은 필수가 아닙니다. 장비 성능은 실제 수업 월드 실행으로 판단합니다.

제출과 문제 해결 원칙

모든 수강생은 GitHub 계정을 준비하고, 자신의 GitHub 아이디를 강사가 지정한 수강생 목록에 등록합니다. Google Slides(구글 슬라이드), Google Classroom(구글 클래스룸), Padlet(패드렛) 중 강사가 지정한 도구에 개인 실습 저장소(repo) 링크를 공유합니다. 강사는 이 공유 목록을 통해 수강생별 저장소와 과제 제출 현황을 관리하고 피드백합니다.

팀 활동이 있는 경우 개인 저장소와 별도로 팀 전용 GitHub 저장소(team repo)를 생성합니다. 팀원과 역할을 README에 기록하고, 팀 저장소 링크도 같은 공유 도구에 등록하여 강사가 팀별 진행 상황과 결과물을 확인할 수 있도록 합니다.

매 실습은 수정한 코드 또는 Git diff, 실행 명령, 예상 결과와 실제 결과, 로그 또는 화면, 실패 원인과 수정 내용을 제출합니다. 재현 절차에는 패키지·실행 항목·파라미터를 적습니다. AI를 사용했다면 질문 요약, 채택한 코드와 직접 확인한 결과를 함께 기록합니다.

Package not found는 빌드와 source, 통신 무응답은 Node·Topic·Type·ROS_DOMAIN_ID·QoS 순서로 확인합니다. RViz2 모델 누락은 Fixed Frame·Robot Description·TF·/joint_states를 확인합니다. Gazebo 센서 누락은 재생 상태·Bridge·/scan을 확인하고, 예상과 다르게 움직이면 Pause 후 원인을 찾습니다.

자료와 검증 범위

- 주교재: 최수길, 『ROS2 입문』, 마담. [책 소개와 전체 목차](#)
- 공개 코드: [장별 코드 대응표](#), [실습 저장소](#)
- 공개 PDF는 1~3장 샘플이며 전체 교재는 포함되지 않습니다. 이 강의계획서는 수업 운영을 위한 별도 문서입니다.
- [저장소 검증 기록](#)은 2026-09-08 기준입니다. 이 계획서 작성 시 모든 Service·Action 시나리오, RViz2 GUI, Gazebo 물리 주행을 새로 검증한 것은 아닙니다. 합성 입력 검사 성공은 실제 주행 성공과 구분합니다.
- GUI 실습이 막힌 경우 Topic 로그·TF 또는 URDF 구조·합성 입력 검사를 대체 증거로 제출하고, GUI·주행 완료 여부는 미검증으로 남깁니다. 강사는 보충 실습 기회를 제공합니다.